

Unitree Week 10

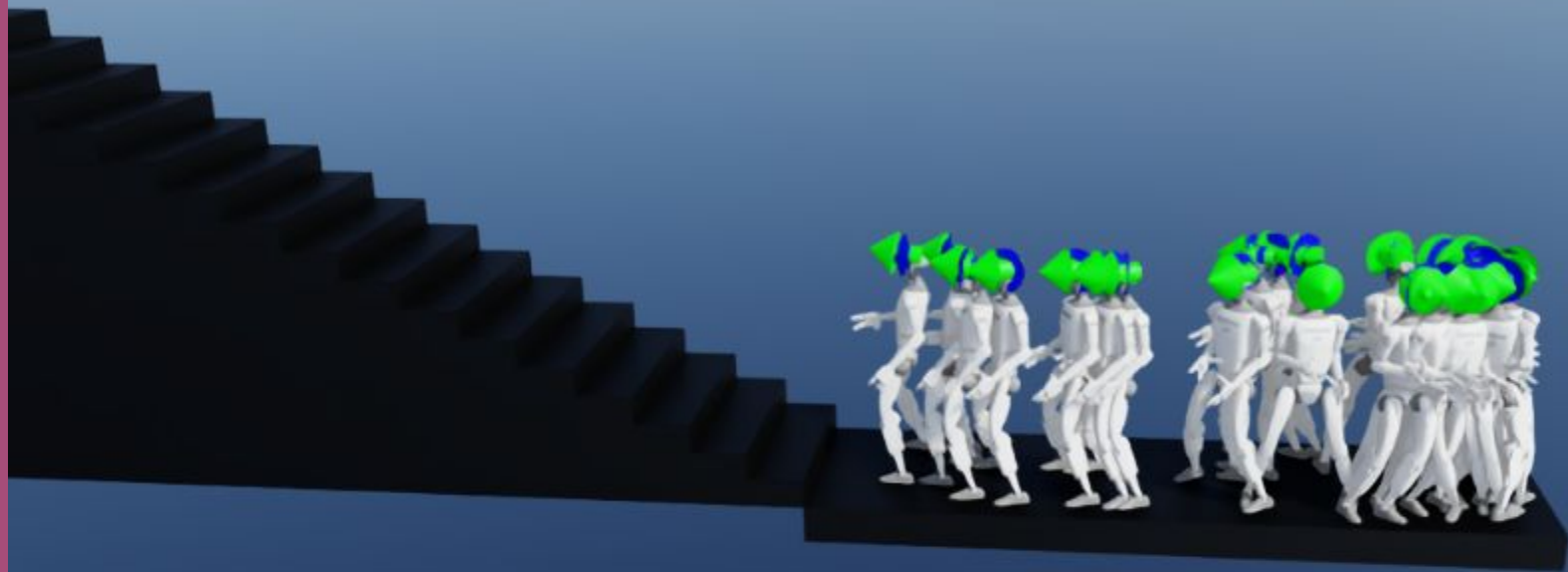
Presentation by Mario Ortiz

This week

- Worked on developing a design for g1 robot walking up stairs.
- Instead of the pyramid terrain, I built a custom staircase terrain for a more controlled training environment.
 - In the pyramid terrain, the g1 robot had various options to survive and obtain rewards (walking down pyramid stairs, walk around one level of the pyramid and therefore not going up or down stairs).
 - With this obstacle, the g1 robot is forced to learn to walk and walk up the stairs in order to obtain rewards.

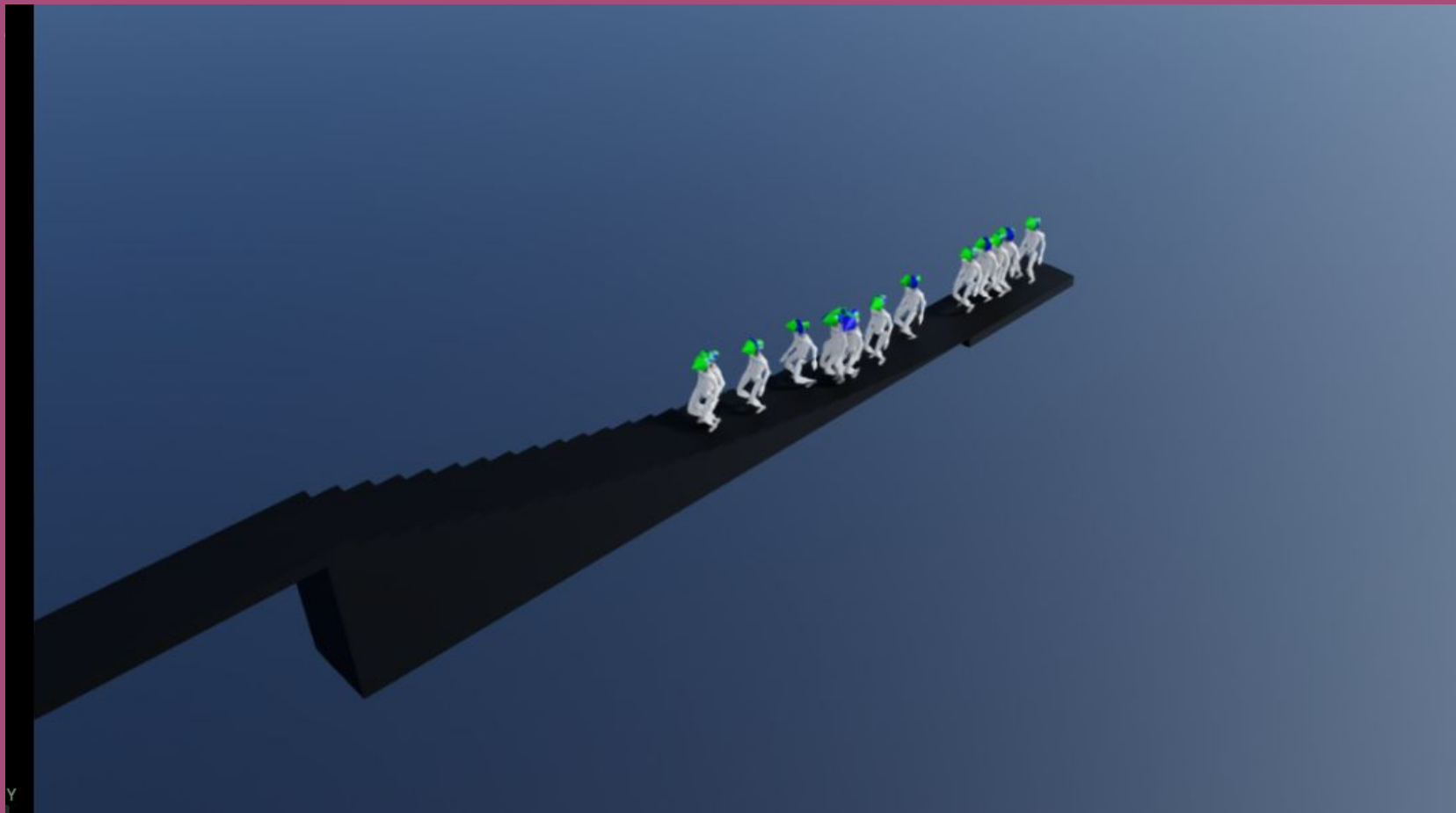
This week





This week

- After the first training attempt, the robots learned to walk until they reached the stairs, then immediately fell, terminating each episode.
- The next idea is to make a staircase in which as steps increase, height of the stair increases, allowing the robot to learn to walk various heights of stairs.
 - Once I can confirm this method teaches the robot to walk up the stairs, I will (in future trainings) make a second staircase with only high level stairs.



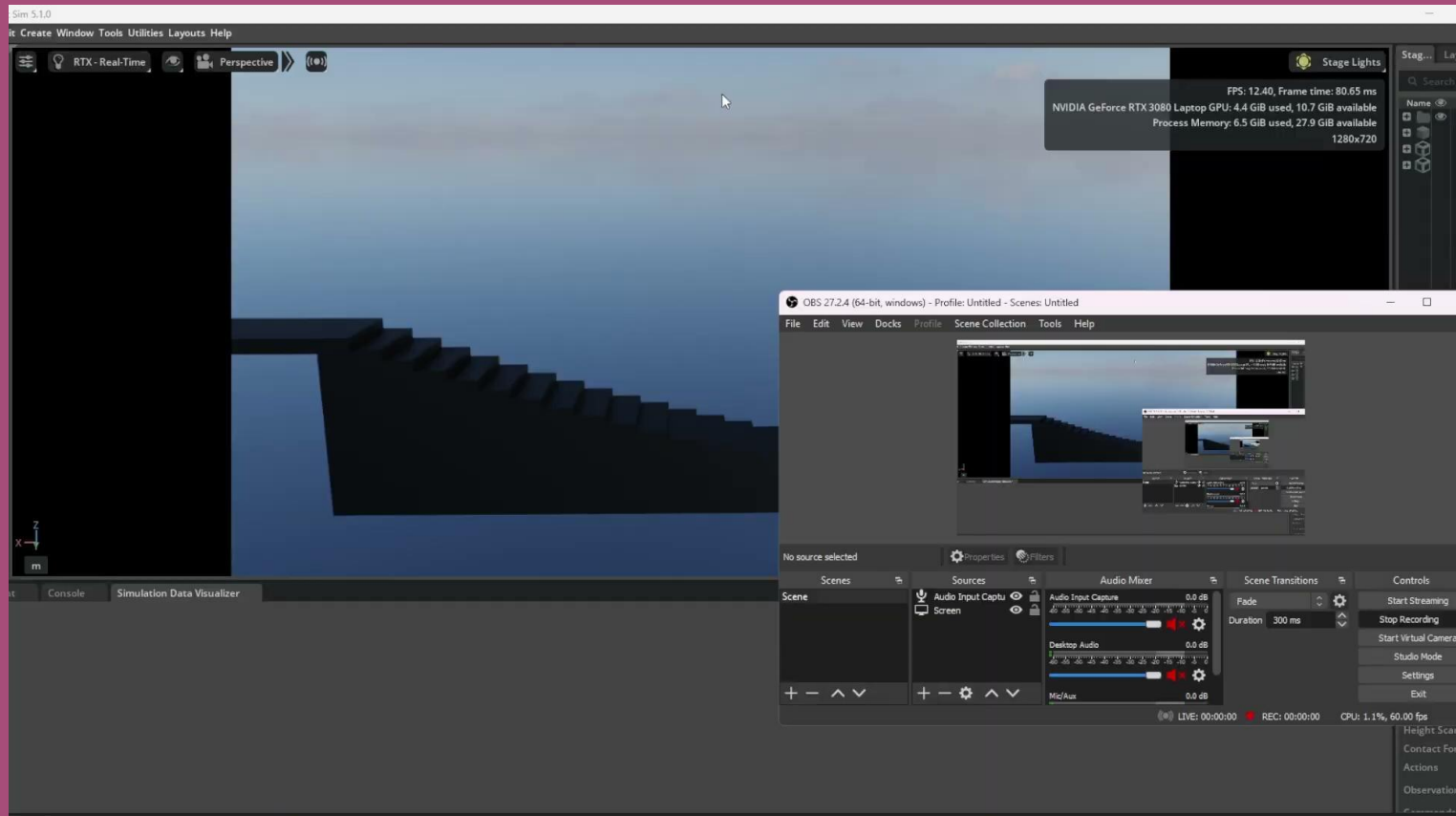
Observations

- The robot's learn to walk up the low-level stairs now. However, once robot's reach halfway up the staircase, the episode ends.

Next Steps

- Found out that max episode length was set to 1000, causing runs to just terminate after it reaches this value.
 - Updated this value in the `stair_env_cfg.py` to double length to ensure it can reach top of the stairs.
 - Found command in `velocity_env_cfg.py` where `self.episode_length_s = 20`, and changed it to 50 (2.5x episode length, allows robot to climb up all stairs).

Video 2



Next steps based on video 2

- Once again, although the robot learned to walk up and down the stairs, it walks in a non-human way.
- Also, it would be preferable if the robot's learned to walk up stairs faster, so I wouldn't have to increase the episode length.

stair_env_cfg.py (lines 1-48)

```
C:\Users\Mario> isaacsim> source> isaacsim_tasks> isaacsim_tasks> manager_based> locomotion> velocity> config> g1>  stair_env_cfg.py>  GIRewards
1  # Copyright (c) 2022-2026, The Isaac Lab Project Developers.
2  # SPDX-License-Identifier: BSD-3-Clause
3
4  import numpy as np
5  import trimesh
6  import torch
7  import math
8  from isaacsim_managers import RewardTermCfg as RewTerm
9  from isaacsim_managers import SceneEntityCfg
10 from isaacsim_managers import TerminationTermCfg as DoneTerm
11 from isaacsim_utils import ConfigClass
12
13 from isaacsim_terrains import MeshPyramidStairsTerrainCfg, TerrainImporterCfg, TerrainGeneratorCfg
14 import isaacsim_tasks.manager_based.locomotion.velocity.mdp as mdp
15 from isaacsim_tasks.manager_based.locomotion.velocity.velocity_env_cfg import LocomotionVelocityRoughEnvCfg, RewardsCfg
16
17 from isaacsim_assets import G1_MINIMAL_CFG
18
19 # Making simple stair terrain:
20 def stair_terrain(difficulty: float, cfg: MeshPyramidStairsTerrainCfg):
21     runway_length = 3.0
22     platform_length = 4.0
23     strip_width = cfg.size[1]
24     num_steps = int(cfg.size[0] // cfg.step_width)
25
26     meshes_list = []
27     def create_box(dims, pos):
28         matrix = trimesh.transformations.translation_matrix(pos)
29         return trimesh.creation.box(dims, matrix)
30
31     # flat start (teaches robot to walk first then approach stairs)
32     meshes_list.append(create_box((runway_length, strip_width, 0.2), (runway_length / 2.0, strip_width / 2.0, -0.1)))
33
34     # increasing the height of stairs
35     current_total_height = 0.0
36     for k in range(num_steps):
37         step_height = 0.02 + (k / num_steps) * 0.13
38         current_total_height += step_height
39         box_pos = (runway_length + (k + 0.5) * cfg.step_width, strip_width / 2.0, current_total_height / 2.0)
40         meshes_list.append(create_box((cfg.step_width, strip_width, current_total_height), box_pos))
41
42     # flat area at end of stairs
43     top_pos = (runway_length + (num_steps * cfg.step_width) + (platform_length / 2.0), strip_width / 2.0, current_total_height - 0.1)
44     meshes_list.append(create_box((platform_length, strip_width, 0.2), top_pos))
45
46     # set origin to the start of the course
47     origin = np.array([0.5, strip_width / 2.0, 0.0])
48     return meshes_list, origin
```

- A simple stair terrain was made using the **trimesh** library. The terrain consists of three parts: a flat runway (so robots initially learn to walk), the staircase with increasing height per step, and another runway on top of the staircase.

stair_env_cfg.py (lines 49-106)

```
50 ##
51 # Pre-defined configs
52 ##
53
54 @configclass
55 class Rewards(RewardsCfg):
56     termination_penalty = RewTerm(func=mdp.is_terminated, weight=-200.0)
57     track_lin_vel_xy_exp = RewTerm(
58         func=mdp.track_lin_vel_xy_yaw_frame_exp,
59         weight=1.0,
60         params={"command_name": "base_velocity", "std": 0.5},
61     )
62     track_ang_vel_z_exp = RewTerm(
63         func=mdp.track_ang_vel_z_world_exp, weight=2.0, params={"command_name": "base_velocity", "std": 0.5})
64
65     feet_air_time = RewTerm(
66         func=mdp.feet_air_time_positive_biped,
67         weight=0.25,
68         params=[
69             "command_name": "base_velocity",
70             "sensor_cfg": SceneEntityCfg("contact_forces", body_names=".*ankle_roll_link"),
71             "threshold": 0.4,
72         ],
73     )
74     feet_slide = RewTerm(
75         func=mdp.feet_slide,
76         weight=0.1,
77         params=[
78             "sensor_cfg": SceneEntityCfg("contact_forces", body_names=".*ankle_roll_link"),
79             "asset_cfg": SceneEntityCfg("robot", body_names=".*ankle_roll_link"),
80         ],
81     )
82     dof_pos_limits = RewTerm(
83         func=mdp.joint_pos_limits,
84         weight=-1.0,
85         params={"asset_cfg": SceneEntityCfg("robot", joint_names=".*ankle_pitch_joint", ".*ankle_roll_joint"))},
86     )
87     joint_deviation_hip = RewTerm(
88         func=mdp.joint_deviation_ll,
89         weight=0.1,
90         params={"asset_cfg": SceneEntityCfg("robot", joint_names=".*hip_yaw_joint", ".*hip_roll_joint"))},
91     )
92     joint_deviation_arms = RewTerm(
93         func=mdp.joint_deviation_ll,
94         weight=0.1,
95         params={"asset_cfg": SceneEntityCfg("robot", joint_names=".*shoulder_.*", ".*elbow_.*")},
96     )
97     joint_deviation_fingers = RewTerm(
98         func=mdp.joint_deviation_ll,
99         weight=0.05,
100         params={"asset_cfg": SceneEntityCfg("robot", joint_names=".*five_.*", ".*three_.*", ".*six_.*", ".*four_.*", ".*zero_.*", ".*one_.*", ".*two_.*")},
101     )
102     joint_deviation_torso = RewTerm(
103         func=mdp.joint_deviation_ll,
104         weight=0.1,
105         params={"asset_cfg": SceneEntityCfg("robot", joint_names="torso_joint")},
106     )
```

- The same rewards/penalties from `rough_env_cfg.py`
- Inherited this class later to add changes

stair_env_cfg.py (lines 107-163)

```
108 @configclass
109 class G1StairEnvCfg(LoComotionVelocityRoughEnvCfg):
110     rewards: G1Rewards = G1Rewards()
111
112     def __post_init__(self):
113         # 1. TERRAIN SETUP
114         self.scene.terrain = TerrainImporterCfg(
115             prim_path="/World/ground", terrain_type="generator",
116             terrain_generator=TerrainGeneratorCfg(
117                 seed=42, size=(12.0, 1.2), border_width=0.8, num_rows=1, num_cols=1,
118                 curriculum=False,
119                 sub_terrains={"stair_strip": MeshPyramidStairsTerrainCfg(
120                     function=stair_terrain, proportion=1.0, step_width=0.40,
121                     step_height_range=(0.95, 0.15)
122                 )},
123             ),
124         )
125
126         # allows to override default commands
127         super().__post_init__()
128
129         # updating max episode length to give g1 enough time to walk up stairs
130         self.episode_length_s = 50.0
131         self.max_episode_length = int(self.episode_length_s / (self.sin.dt * self.decimation))
132
133         self.scene.robot = G1_MINIMAL_CFG.replace(prim_path="/ENV_REPLACE_NS/Robot")
134         self.scene.height_scanner.prim_path = "/ENV_REPLACE_NS/Robot/torso_link"
135
136         # force robot to spawn looking at the staircase
137         self.events.reset_base.params["pose_range"]["yaw"] = (0.0, 0.0)
138         self.events.reset_base.params["pose_range"]["x"] = (-0.1, 0.1)
139         self.events.reset_base.params["pose_range"]["y"] = (-0.1, 0.1)
140
141         self.terminations.base_height = DoneTerm(func=mdp.root_height_below_minimum, params={"minimum_height": -0.2})
142
143         # rewards and events (prelarily same as rough_env_cfg.py)
144         self.events.push_robot = None
145         self.events.add_base_mass = None
146         self.events.base_com = None
147         self.events.reset_robot_joints.params["position_range"] = (1.0, 1.0)
148
149         self.rewards.lin_vel_z_12.weight = 0.0
150         self.rewards.undesired_contacts = None
151         self.rewards.flat_orientation_12.weight = -1.0
152         self.rewards.action_rate_12.weight = -0.005
153         self.rewards.dof_acc_12.weight = -1.25e-7
154         self.rewards.dof_acc_12.params["asset_cfg"] = SceneEntityCfg("robot", joint_names=["*_hip_*", "*_knee_joint"])
155         self.rewards.dof_torques_12.weight = -1.5e-7
156         self.rewards.dof_torques_12.params["asset_cfg"] = SceneEntityCfg("robot", joint_names=["*_hip_*", "*_knee_joint", "*_ankle_*"])
157
158         # Forces robot to move in 1 direction (towards the staircase)
159         self.commands.base_velocity.ranges.lin_vel_x = (0.4, 0.6)
160         self.commands.base_velocity.ranges.lin_vel_y = (0.0, 0.0)
161         self.commands.base_velocity.ranges.ang_vel_z = (0.0, 0.0)
162
163         self.terminations.base_contact.params["sensor_cfg"].body_names = "torso_link"
164
165 @configclass
166 class G1stairEnvCfg_PLAY(G1StairEnvCfg):
167     def __post_init__(self):
168         super().__post_init__()
169         self.scene.num_envs = 32
170         self.commands.base_velocity.ranges.lin_vel_x = (0.5, 0.5)
```

- Same code as the initial rough_env_cfg.py with some tweaks:
- Yaw lock is set to (0.0, 0.0), ensuring robots always face the stairs once simulation begins.
- Spawn range also decreased (-0.1, 0.1), ensuring all robot's spawn in the same position.
- root_height_below_minimum: ensures episode terminates once robot's fall off the terrain.
- dof_acc_12: tweaked to focus primarily on hip and knee joints.
- dof_torques_12: tweaked to focus on hip, knee, and ankles.
- flat_orientation_12: penalty of -1 to ensure robot stands upright.
- lin_vel_z_12: set to 0 as we don't want to punish g1 for moving up and down (since we are walking up and down stairs).
- lin_vel_xy_exp: robot will now only get this reward based on x position (rewards for moving towards stairs, not for moving left and right).
- ang_vel_z_exp: set to 0 so it doesn't get rewarded for angling it's body downwards (in earlier trainings, instead of moving up the stairs, the robots learned to maximize rewards by angling its body downwards for z position rewards).

Other trainings

- I have started working on a new training code where the g1 robot learns to walk while holding a box. The idea is that the g1 will learn to pick up the box and then walk with the box.

Important logs

- Walks half of staircase (max 1000 episode length):
2026-02-24_14-03-30
- Walks up entire staircase and walks back down some stairs (max 2500 episode length): 2026-02-2